

Sound Localising Methods

1. Model the incoming sound

First we have to model the incoming sound somehow.

$$x_i(t) = s(t) * h_i(t) + n_i(t) \quad (1)$$

Where:

- $x_i(t)$ is the signal captured by the i -th microphone.
- $s(t)$ is the source signal.
- $h_i(t)$ is the room response. It consists of the time difference that the sound has to travel and all other reverberations. (Convolution can apply these features at the same time).
- $n_i(t)$ is the noise captured by the i -th microphone.

2. Cross-correlation

Then basically we'll calculate cross-correlation (later we'll use tricks but the core is still a cross-correlation problem).

So:

$$R_{ij}(\tau) = \int_0^T x_i(t)x_j(t + \tau) dt \quad (2)$$

In theory we could do this but when we're working with computers we should apply discrete methods.

$$R_{ij}[\tau] = \sum_{k=0}^N x_i[k]x_j[k + \tau] \quad (3)$$

It is a cross correlation that gives us back how the two signals match as a function of τ , then we can select the τ that maximizes this function and say that's the time difference of arrival.

3. Basic CC, GCC-PHAT

But we can do it in more sophisticated way by looking at these events in frequency domain.

But before we do that it's worth noticing that the computation of cross-correlation and convolution is really similar. And convolution in time domain becomes multiplication in frequency domain.

So:

$$R_{ij}(\tau) = x_i(t) \star x_j(-t) \quad (4)$$

(The minus sign becomes complex conjugation).

$$G_{ij}(\omega) = X_i(\omega)X_j^*(\omega) \quad (5)$$

This is a key part of all the following algorithm. From there the methods differ. In basic CC we just transform $G_{ij}(\omega)$ back to time domain. And select the τ that maximize $R_{ij}(\tau)$.

In GCC-PHAT before we transform the signal back to get the TDOA we whiten the signals by removing the magnitude. This makes the method more precise.

So, we'll end up with $\frac{G_{ij}}{|G_{ij}|}$ and then we transform it back to time domain.

$$R_{ij}^{PHAT}(\tau) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \frac{X_i(\omega)X_j^*(\omega)}{|X_i(\omega)X_j^*(\omega)|} e^{i\omega\tau} d\omega \quad (6)$$

So by doing this we'll get a pulse at the estimated TDOA.

So with basic CC and GCC-PHAT we can get the time delays for every pair of mics.

After we got the time delays we can use basic trigonometry to get the estimated direction. We assume that the sound source is far away from the mics. Than we estimate that the wavefront of the sound, near the mics is a straight line. By doing this we can get the DoA by doing some basic trigonometry. However, using only one pair of mics won't give us a full solution. In space time delays determine a cone of possible directions where the sound can come from. If we have the results from two different mic pairs than it determines two cones. It's relatively easy to prove that the intersection of these cones determine two straight lines in space. One above the mic array and one under. These will be the lines corresponding to the direction of arrival. (Actually the TDOA defines a hyperboloid but we can estimate it with cones assuming far-field scenario.)

4. SRP - PHAT

SRP consists of two parts. First, Steered Response Power. Second, Phase transform. We demonstrated the Phase transform above.

Steered Response Power asks the question: What would we sense if sound were coming from a point in 3D space $q = (x, y, z)$? If we have this imaginary

sound source at q , then we can compute the time required to reach the i -th mic let's name this $\Delta_i(q)$. We add up the modified signals to model the interference produced by the time delays.:

$$y(t, q) = \sum_{i=0}^{M-1} x_i(t - \Delta_i(q))$$

(M is the num of mics)

(We can reverse this and think about this process as; what happens if we sum up the signals coming from the mics with the delays at q , and observe what type of interferences do we get.)

We want to find the q that maximizes $y(t, q)$. We can't really work with $y(t, q)$. Solving this issue we could average $y(t, q)$ over a time period, but frequent jumps from positive to negative values appear in every sound recording, so the signal would average out. Solving this we square the signal before the time average:

$$P(q) = \frac{1}{T_2 - T_1} \int_{T_1}^{T_2} y^2(t, q) dt$$

or using discrete case:

$$P(q) = \frac{1}{N} \sum_{n=0}^{N-1} y^2(n, q)$$

Let's denote it: $P(q) = E[y^2(t, q)]$
expanding:

$$P(q) = E \left[\left(\sum_{i=0}^{M-1} x_i(t - \Delta_i(q)) \right) \left(\sum_{j=0}^{M-1} x_j(t - \Delta_j(q)) \right) \right]$$

$$P(q) = \sum_{i,j=0}^{M-1} E[x_i(t - \Delta_i(q))x_j(t - \Delta_j(q))]$$

Let's define:

$$R_{ij}(\tau) = E[x_i(t)x_j(t - \tau)]$$

Where:

$$\tau = \Delta_j(q) - \Delta_i(q)$$

$$P(q) = \sum_{i,j=0}^{M-1} R_{ij}(\tau)$$

So we arrived back to cross-correlation. Now we can apply PHAT:

$$P(q) = \sum_{i,j=0}^{M-1} R_{ij}^{\text{PHAT}}(\tau)$$

We compute unnecessary things (twice the correlations when $i \neq j$, and cross-correlation with the signal itself too, when $i = j$), let's simplify it because we only care about the maximum of $P(q)$

$$P'(q) = \sum_{i=0}^{M-1} \sum_{j=i+1}^{M-1} R_{ij}^{\text{PHAT}}(\tau)$$

So, we calculate τ from q and then from q we calculate $P'(q)$. We do that for many points in space (q) and the point that gives the maximum value for $P'(q)$ will be selected as the estimated sound source position. As you can see this algorithm is computationally heavy, but it estimates the DoA more accurately than many of its competitors. And it can handle echos and noise better.

Neuromorphic hardware

All the algorithms discussed earlier are precise mathematical methods designed for digital computers. While they are tunable and highly accurate, they demand significant computing power. An alternative approach is to use an analog, biologically-inspired method that avoids traditional digital computing. Coincidence detection using Leaky Integrate-and-Fire (LIF) neurons provides such a method for determining the Direction of Arrival (DoA).

The core concept consists of two mechanisms. First, input signals are converted into voltage spikes. When a spike hits a neuron, its amplitude adds to the neuron's current excitation (voltage) level. If this cumulative excitation level exceeds a specific threshold, the neuron fires an output spike. This serves as the 'integrate' phase. Second, the excitation level of the neuron continuously decays exponentially over time unless another spike arrives. This is the 'leaky' property.

These LIF neurons model actual biological neurons more accurately than standard artificial neural networks (ANNs). It is heavily hypothesized that human and animal sound localization relies on highly similar neuronal coincidence detection mechanisms in the brain stem.

To leverage these properties, we introduce specific, hard-coded time delays to these signals. Assuming we have n microphones, if n delayed sound spikes reach a neuron nearly simultaneously, their integration overcomes the threshold of the neuron, and it fires. By mapping different delay combinations to specific spatial angles, we can detect the DoA based entirely on which neuron fires first.

To verify this method, we implemented it in Python using the Brian 2 library, which is specifically designed to simulate spiking neural hardware. First, we created an ideal acoustic simulation. This assumes the sound source is in the far-field, producing perfectly parallel wavefronts without noise or echoes. We

fed the physical geometric parameters of our ReSpeaker array into this simulation. It is crucial to tune the neuronal parameters carefully; otherwise, the coincidence overlap fails. To find the optimal parameters, we conducted a grid search algorithm. Through this, we successfully tuned the exponential decay constant (τ_{leaky}) and the firing threshold voltage to perfectly estimate the DoA under ideal conditions.

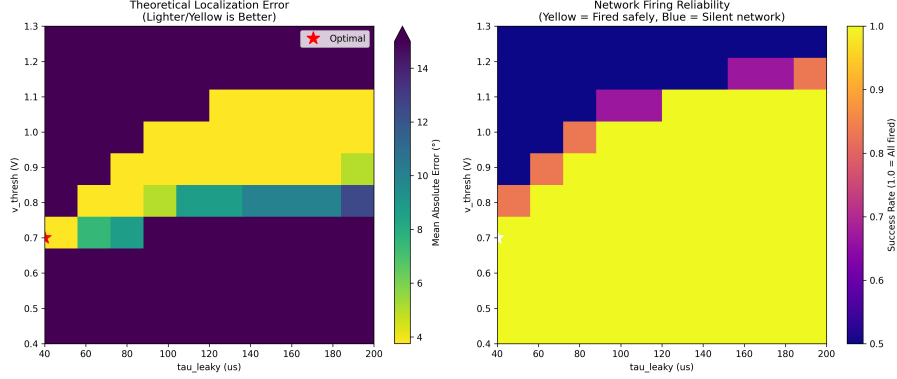


Figure 1: Parameter optimization grid search mapping the theoretical localization error limits, demonstrating the optimal parameter selection.

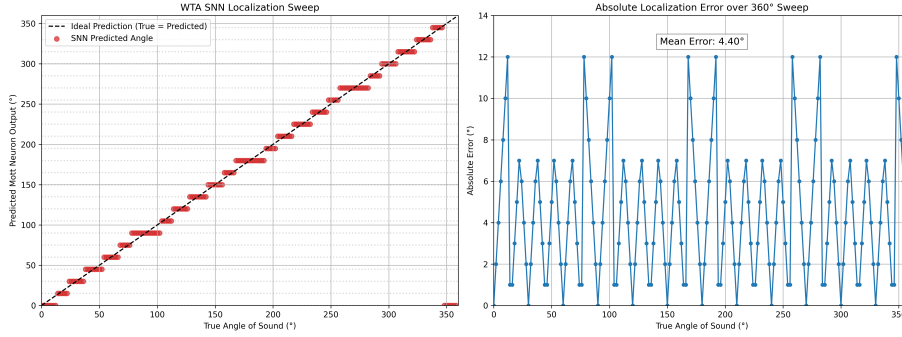


Figure 2: Localization accuracy evaluated across a full 360° sweep using the optimized LIF parameters in the ideal acoustic simulation.

While the algorithm performed perfectly in an ideal scenario, real acoustic environments are far from optimal. To verify the viability of this method in real-world conditions, I created a simulation that processes digital audio data recorded directly from a physical sound source using the ReSpeaker mic array. First, the four raw microphone channels were mathematically upsampled to a significantly higher frequency using SciPy. This was necessary because the original 16 kHz sampling rate suffered from digital quantization jitter, lacking the

sub-millisecond physical precision required to accurately run the SNN coincidence math on such a small hardware array.

Next, the program implements a dynamic amplitude threshold: during silence, no spikes are generated. If the audio crosses this loudness threshold, an algorithm isolates the peak and a precise discrete spike is generated to represent the leading sound wavefront. These exact physical spikes are then routed through the spatial delay matrix and sent into the LIF neuron layer to execute the localization.